Controller

```
using Day12Api.Interfaces;
using Day12Api.Models;
using Day12Api.Services;
using Microsoft.AspNetCore.Mvc;

namespace Day12Api.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class NewMovieController : ControllerBase
    {
        private readonly INewMovieService _movieService;

        public NewMovieController(INewMovieService movieService)
        {
            _movieService = movieService;
        }

        [HttpPost("AddDirector")]
        public IActionResult AddDirector(MovieDirector director)
        {
            var result = _movieService.AddDirector(director);
            return Ok(result);
        }

        [HttpPost("AddNewMovies")]
        public IActionResult AddNewMovies(MovieAndDirectorEntry movieAndDirector)
        {
            var result = _movieService.AddNewMovie(movieAndDirector);
            if (result == "Director Not Found") return NotFound(result);
            return Ok(result);
        }

        [HttpPost("AddNewMovieSales")]
        public IActionResult AddSales(SalesDto salesDto)
        {
            var result = _movieService.AddMovieSales(salesDto);
            if (result == "Movie Not Found") return NotFound(result);
            return Ok(result);
        }

        [HttpGet("GetAllNewMovies")]
```

```
        public IActionResult GetAllNewMovies()
        {
            var movies = _movieService.GetAllNewMovies();
            return Ok(movies);
        }

        [HttpGet("FetchAllNewMoviesByAuthor")]
        public IActionResult GetAllNewMoviesByAuthor(string director)
        {
            var movies = _movieService.GetAllNewMoviesByAuthor(director);
            return Ok(movies);
        }

        [HttpGet("FetchAllNewMoviesBySales")]
        public IActionResult GetAllNewMoviesBySales()
        {
            var movies = _movieService.GetAllNewMoviesBySales();
            return Ok(movies);
        }

        [HttpGet("GetSalesByAuthor")]
        public IActionResult GetSalesByAuthor(string name)
        {
            var sales = _movieService.GetSalesByAuthor(name);
            return Ok(sales);
        }
    }
}
```

Interface

```
using Day12Api.Models;

namespace Day12Api.Interfaces
{
    public interface INewMovieService
    {
        string AddDirector(MovieDirector director);
        string AddNewMovie(MovieAndDirectorEntry movieAndDirector);
        string AddMovieSales(SalesDto salesDto);
        List<NewMovie> GetAllNewMovies();
        object GetAllNewMoviesByAuthor(string directorName);
        object GetAllNewMoviesBySales();
        object GetSalesByAuthor(string directorName);
```

```
    }
}
```

Services

```
using Day12Api.Context;
using Day12Api.Interfaces;
using Day12Api.Models;
using Microsoft.EntityFrameworkCore;

namespace Day12Api.Services
{
    public class NewMovieService : INewMovieService
    {
        private readonly MyAppDbContext _context;

        public NewMovieService(MyAppDbContext context)
        {
            _context = context;
        }

        public string AddDirector(MovieDirector director)
        {
            _context.movieDirector.Add(director);
            _context.SaveChanges();
            return "Director Added Successfully";
        }

        public string AddNewMovie(MovieAndDirectorEntry movieAndDirector)
        {
            var director = _context.movieDirector.FirstOrDefault(x => x.DirectorName ==
movieAndDirector.DirectorName);
            if (director == null) return "Director Not Found";

            var movie = new NewMovie
            {
                Title = movieAndDirector.Title,
                Price = movieAndDirector.Price,
                DirectorId = director.DirectorId
            };

            _context.newMovies.Add(movie);
            _context.SaveChanges();
            return "Movie Added Successfully";
```

```
}

public string AddMovieSales(SalesDto salesDto)
{
    var movie = _context.newMovies.FirstOrDefault(x => x.MovieId == salesDto.MovieId);
    if (movie == null) return "Movie Not Found";

    var sale = new Sales
    {
        MovieId = salesDto.MovieId,
        NumberOfDays = salesDto.NumberOfDays,
        Year = salesDto.Year
    };

    _context.movieSales.Add(sale);
    _context.SaveChanges();
    return "Sales Details Added Successfully";
}

public List<NewMovie> GetAllNewMovies()
{
    return _context.newMovies.Include(x => x.director).ToList();
}

public object GetAllNewMoviesByAuthor(string directorName)
{
    return _context.newMovies
        .Include(x => x.director)
        .Where(y => y.director.DirectorName == directorName)
        .Select(z => new
        {
            z.MovieId,
            z.DirectorId,
            z.Title,
            z.director.DirectorName
        }).ToList();
}

public object GetAllNewMoviesBySales()
{
    var result = from ms in _context.movieSales
                 join nm in _context.newMovies on ms.MovieId equals nm.MovieId
                 join dir in _context.movieDirector on nm.DirectorId equals dir.DirectorId
                 orderby ms.NumberOfDays ascending
```

```
                select new
                {
                    nm.Price,
                    nm.Title,
                    dir.DirectorName,
                    ms.Year,
                    ms.NumberOfDays
                };
            return result.ToList();
        }

        public object GetSalesByAuthor(string directorName)
        {
            var result = from ms in _context.movieSales
                         join nm in _context.newMovies on ms.MovieId equals nm.MovieId
                         join dir in _context.movieDirector on nm.DirectorId equals dir.DirectorId
                         where dir.DirectorName == directorName
                         select new
                         {
                             nm.Price,
                             nm.Title,
                             dir.DirectorName,
                             ms.Year,
                             ms.NumberOfDays
                         };
            return result.ToList();
        }
    }
}
```

Program.cs

```
using Day12Api.Interfaces;
using Day12Api.Models;
using Day12Api.Services;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using System.Security.Claims;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
```

```
builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();




builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new() { Title = "SampleApi", Version = "v1" });

    // ?? Add JWT Bearer Authentication to Swagger
    c.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = Microsoft.OpenApi.Models.SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = Microsoft.OpenApi.Models.ParameterLocation.Header,
        Description = "Enter 'Bearer' [space] and then your valid token.\nExample: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6..."
    });

    c.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
    {
        {
            new Microsoft.OpenApi.Models.OpenApiSecurityScheme
            {
                Reference = new Microsoft.OpenApi.Models.OpenApiReference
                {
                    Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            new string[] {}
        }
    });
});

builder.Services.AddTransient<INewMovieService, NewMovieService>();

var sql_section = builder.Configuration.GetSection("Sql_Connection");
string connectionString = sql_section["conn_str"];
```

```
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));
var jwt = builder.Configuration.GetSection("Jwt");
var issuer = jwt["Issuer"];
var audience = jwt["Audience"];
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt["key"]));

builder.Services.AddAuthentication(options =>
{
   options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
   options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;

}).AddJwtBearer(options =>
{
   options.TokenValidationParameters = new TokenValidationParameters
   {
      ValidIssuer = issuer,
      ValidAudience = audience,
      IssuerSigningKey = key,
      ValidateIssuer = true,
      ValidateAudience = true,
      ValidateIssuerSigningKey = true,
      ValidateLifetime = true,
      ClockSkew = TimeSpan.Zero,
      RoleClaimType = ClaimTypes.Role,
      NameClaimType = ClaimTypes.NameIdentifier
   };

});

builder.Services.AddDbContext<Day12Api.Context.MyAppDbContext>
   (x => x.UseSqlServer(connectionString));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
   app.UseSwagger();
   app.UseSwaggerUI();
}

app.UseHttpsRedirection();
```

app.UseAuthorization();

app.MapControllers();

app.Run();

Curl

```
curl -X 'GET' \
  'https://localhost:7201/NewMovie/FetchAllNewMoviesBySales' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:7201/NewMovie/FetchAllNewMoviesBySales
```

Server response

Code	Details
200	Response body


```
[
  {
    "price": 500,
    "title": "Coolie",
    "directorName": "Lokesh",
    "year": "12",
    "numberOfDays": 12
  },
  {
    "price": 300,
    "title": "Vikram",
    "directorName": "Lokesh",
    "year": "1001",
    "numberOfDays": 20
  },
  {
    "price": 500,
```